



Agencia de Habilidades
para el Futuro

<Talento
Tech/>

FRONT-END JS

**Clase n° 15 | API y Procesamiento de Datos en
E-commerce**

¡Les damos la bienvenida!



 Vamos a comenzar a grabar la clase.

Índice

1

15. API y Procesamiento de Datos

- Asincronía Desarrollo de un proyecto integrador que combine HTML, CSS, y JavaScript
- Consumo de API REST
- Incorporación y procesamiento de los datos en nuestro HTML
- Incorporación de buenas prácticas de accesibilidad y SEO
- Presentación del proyecto final

2

16. Revisión final y despedida

- Presentación y revisión de los proyectos finales
- Feedback personalizado a cada proyecto
- Comparación con proyectos profesionales
- Despedida y recomendaciones para el futuro

Introducción al Consumo de API REST

API REST

¿Qué es una API REST?

1

Comunicación estructurada

Permite que dos sistemas intercambien información de forma eficiente.

2

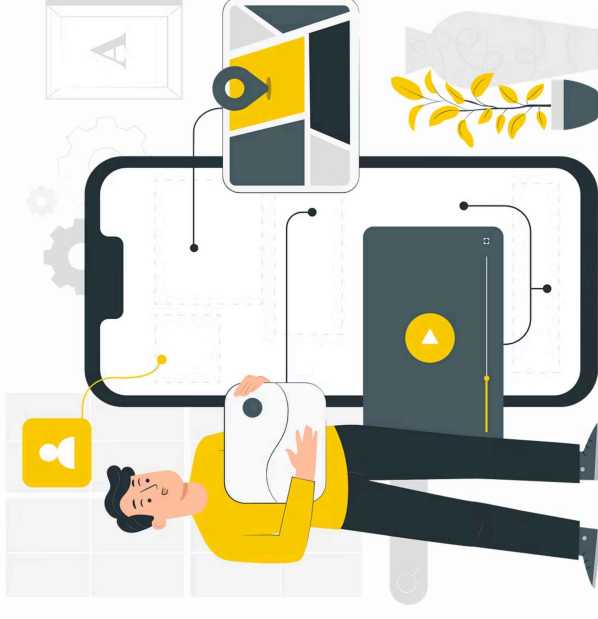
Obtención de datos

Las aplicaciones pueden obtener y enviar datos a un servidor.

3

Experiencias dinámicas

Facilita la creación de contenido en tiempo real.



Ejemplos de Uso de APIs en Empresas de Tecnología



Redes Sociales

Facebook, Twitter e Instagram ofrecen acceso a perfiles y publicaciones.



E-commerce

Amazon y eBay facilitan la integración de productos en tiendas externas.



Mapas y Geolocalización

Google Maps y OpenStreetMap proporcionan información geográfica en tiempo real.



Pago y Autenticación

PayPal, Stripe, y MercadoPago ofrecen APIs para gestionar pagos.



Entretenimiento

Spotify y YouTube permiten integrar contenido musical o audiovisual.

¿Por qué se usa JSON en lugar de XML?

Ligereza

Compatibilidad

Legibilidad

JSON es más ligero y fácil de leer que XML, permitiendo transferencias más rápidas.

JSON es más compatible con JavaScript, convirtiéndose directamente en objetos.

JSON es menos verboso, haciendo que el código sea más claro.

Consumo de APIs con `fetch()`

Consumo de APIs con `fetch()`

1 Solicitudes HTTP

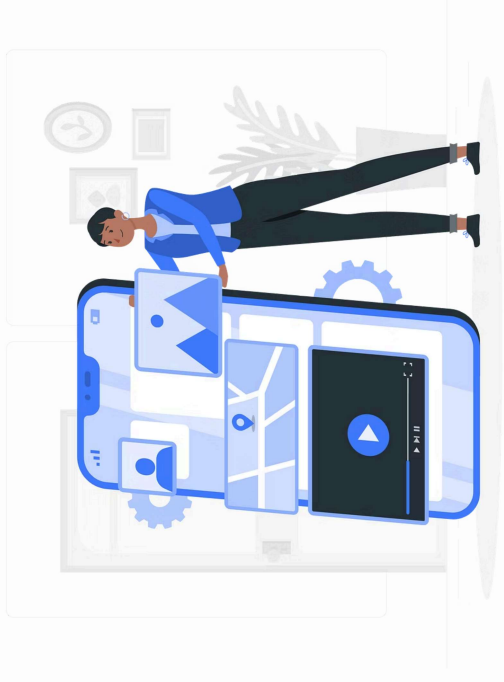
`fetch()` permite realizar solicitudes asíncronas a una API.

2 Experiencia mejorada

Obtiene datos sin recargar la página, mejorando la experiencia del usuario.

3 Procesamiento de respuesta

Convierte los datos recibidos en objetos JSON para su fácil manejo.



Ejemplo básico de `fetch()`

```
fetch('https://fakestoreapi.com/products')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Procesamiento de la Respuesta en JSON

Usamos `response.json()` para convertir los datos en un objeto JSON, lo cual facilita el procesamiento en JavaScript.

```
fetch('https://fakestoreapi.com/products')
  .then(response => response.json())
  .then(data => {
    // Aquí se procesan los datos obtenidos y se integran en el DOM
  });
```

Manejo de Errores con `.catch()` y Códigos 400/500

⚠ Cuando una solicitud a la API falla, es esencial manejar el error para que el usuario reciba un mensaje claro.

Errores Comunes:

1 Errores 400

Problemas con la solicitud, como parámetros incorrectos o falta de permisos. Ejemplo: 404 (No Encontrado), 401 (No Autorizado).

2 Errores 500

Problemas en el servidor. Estos errores son más difíciles de predecir, ya que ocurren en el lado del servidor.

Códigos de Error HTTP Comunes

4XX Client Error	
400	Bad Request
401	Unauthorized
402	Payment Required
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
407	Proxy Authentication Required
408	Request Timeout

5XX Server Error	
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported
506	Variant Also Negotiates
507	Insufficient Storage
508	Loop Detected
510	Not Extended
511	Network Authentication Required
599	Network Connect Timeout Error

Ejemplo de Manejo de Errores

```
fetch('https://fakestoreapi.com/products')
  .then(response => {
    if (!response.ok) {
      throw new Error(`HTTP error! Status: ${response.status}`);
    }
    return response.json();
  })
```

```
})  
.then(data => {  
  // Procesamiento de datos  
})  
.catch(error => {  
  console.error('Error en la comunicación con la API:', error);  
  // Aquí podrías mostrar un mensaje de error al usuario  
});
```

Renderizado Dinámico de Productos

Renderizado Dinámico de Productos

1 Crear contenedor HTML

En el HTML, necesitamos un contenedor para mostrar los productos de forma dinámica:

```
<div id="productos-container"></div>
```

2 Procesar datos

Con los datos obtenidos de la API, usamos JavaScript para generar elementos HTML que representen cada producto (imagen, título, precio, botón de compra)

3 Integrar en el DOM

Insertamos

- Ejemplo código

Buenas Prácticas de Accesibilidad y SEO

Buenas Prácticas de Accesibilidad



Etiquetas Alt

Usar atributos `alt` en imágenes para mejorar la experiencia de usuarios con discapacidades visuales.



Navegación por Teclado

Asegurar que los elementos interactivos sean accesibles mediante teclado.



Estructura Semántica

Utilizar etiquetas HTML semánticas para una mejor interpretación del contenido. (`<header>`, `<main>`, `<section>`, `<footer>`).

Optimización SEO en el Proyecto

1

Metaetiquetas

Las etiquetas `<meta>` en el `<head>` mejoran la visibilidad en buscadores.

2

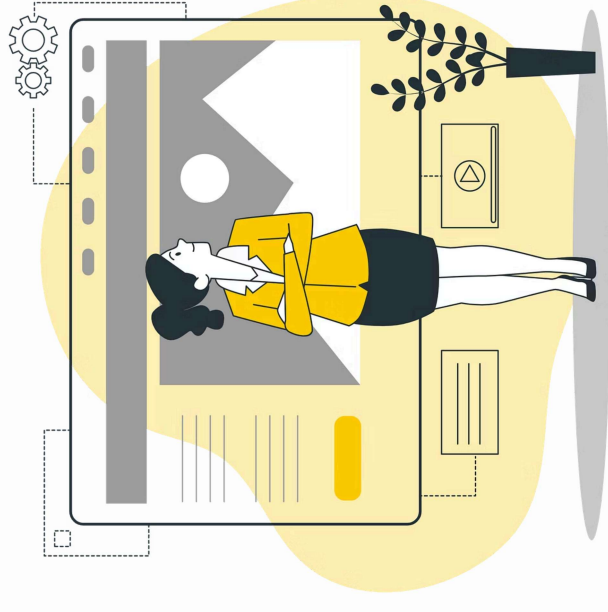
Contenido Optimizado

Asegurar que las descripciones y nombres de productos sean claros y relevantes.

3

Encabezados Lógicos

Usar encabezados (`<h1>`, `<h2>`, `<h3>`) en una estructura jerárquica para que los motores de búsqueda puedan entender la relevancia del contenido.



Entrega de Proyecto Final

Proyecto Final de E-commerce

Desarrollarás una página web completa, que combine todos los conocimientos adquiridos a lo largo del curso. El proyecto consistirá en la creación de un sitio web de **e-commerce** dinámico e interactivo, que consuma datos de una API REST para mostrar productos, y permita a los usuarios añadir productos a un carrito de compras.

Pasos a seguir:

- **HTML:** Uso de etiquetas semánticas para organizar la página.
- **CSS:** Implementación de un diseño responsivo y atractivo usando Bootstrap y Flexbox.
- **JavaScript:** Integración de una API REST para obtener datos y renderizar productos en el DOM, además de la funcionalidad de un carrito de compras usando `localStorage`.
- **Accesibilidad y SEO:** Implementar prácticas que mejoren la experiencia del usuario y optimicen la página para los motores de búsqueda.

Puntos Clave para Revisión y Entrega

- **Subida del Proyecto:** Debe estar disponible en GitHub Pages o Netlify para facilitar su acceso.
- **Control de versiones:** Mantener un historial de commits detallado para documentar cada avance.
- **Presentación:** El archivo [README.md](#) debe incluir una descripción del proyecto, las tecnologías usadas, instrucciones de instalación y cualquier detalle relevante.

Formato de Entrega

1 Carpeta en Drive

Crear una carpeta en drive (pública) que contenga los archivos y carpetas que conforman tu proyecto.

2 Link de Entrega

Los links deberán ser entregados en el apartado de “Pre-Entrega de Proyecto” en el Campus Virtual.

3

Hosting

Compartir el link de Github Pages o subida a servidor.

Requisitos para la entrega



Entrega de Proyecto Final

Obligatorio

A partir de la Clase N° 15 tendrás 7 días de corrido para realizar la entrega en el campus virtual

1. Estructura Básica de HTML.

- **Estructura semántica:** El HTML debe estar dividido en las etiquetas semánticas principales: **header**, **nav**, **main**, **section**, **footer**.

README.md: Incluir un archivo que explique brevemente el propósito de la página.

2. Formulario de Contacto.

Formulario funcional: Crear un formulario de contacto con campos para **nombre**, **correo electrónico** y **mensaje**, utilizando **Formspree** para manejar el envío de datos.

Entrega de Proyecto Final

Obligatorio

3. Estilos Básicos Aplicados con CSS

- **Archivo styles.css:** El proyecto debe contar con un archivo CSS externo que incluya:
 - Estilos básicos aplicados a las secciones de **header**, **footer** y **lista de navegación**.
 - **Fuentes de Google Fonts** correctamente implementadas.

Propiedades de **background** aplicadas en alguna sección de la página (color, imagen, degradado, etc.).

4. Diseño Responsivo con Flexbox y Grid

- **Sección "Productos":** Organizada en **cards** de forma responsiva utilizando **Flexbox**.
- **Sección "Reseñas":** Organizada utilizando **Grid**, con una distribución lógica y estética.
- **Sección "Contacto":** Debe ser responsiva mediante el uso de **Media Queries** para adaptarse a diferentes tamaños de pantalla.



Pre-Entrega de Proyecto

Obligatorio

5. Contenido Multimedia y Navegación

- **Multimedia:** deberá incluir archivos multimedia (imagenes, video o iframe) correctamente integrado en la página.

Lista de navegación: Implementar una **lista desordenada** con enlaces que simulen una navegación interna (Inicio, Productos, Contacto, etc.).

6. Subida del Proyecto.

El proyecto debe estar subido a un **hosting gratuito** (Netlify o GitHub Pages), con una **URL funcional** para visualizar el sitio.



Pre-Entrega de Proyecto

Obligatorio

7. JavaScript

- **Script.js:** deberá incluir un archivo Debes crear un archivo script.js para manejar toda la interactividad de la página.

Asegúrate de enlazarlo correctamente en tu archivo HTML.

- **DOM:** Implementa funciones para validar formularios (ej., campos requeridos y formato de correo).

Usa JavaScript para manipular elementos del DOM, por ejemplo, actualizar el carrito y mostrar mensajes al usuario

- **Fetch Api**

Consume datos desde una API REST usando fetch.

Muestra los productos obtenidos de la API en la página en forma de tarjetas (cards).

- **Visualización de Productos:**

Cada producto debe tener su imagen, título y precio, mostrando una lista atractiva para el usuario.



Pre-Entrega de Proyecto

Obligatorio

8. Carrito de compras dinámico

- **Agregar Productos al Carrito:** Implementa un carrito de compras donde los usuarios puedan añadir productos desde las tarjetas.
- **Uso de `localStorage` o `sessionStorage`:** Guarda el estado del carrito en `localStorage` o `sessionStorage` para que no se pierda al actualizar o cerrar la página.

Contador Dinámico: Muestra el número total de productos en el carrito y asegúrate de actualizarlo en tiempo real.

9. Edición y visualización del carrito

- **Visualización de Productos en el Carrito:** Muestra una lista de productos añadidos al carrito, incluyendo cantidad, precio y total.
- **Edición de Cantidades y Eliminación de Productos:** Implementa funciones para que el usuario pueda editar la cantidad de cada producto o eliminarlo del carrito.

Total Dinámico: Actualiza el total de la compra cada vez que se modifiquen los productos en el carrito.

Pre-Entrega de Proyecto

Obligatorio

10. SEO & Accesibilidad

Buenas Prácticas de Accesibilidad:

- Usa alt en las imágenes para mejorar la accesibilidad.
- Asegúrate de que se pueda navegar fácilmente con el teclado.

SEO Básico:

- Usa metaetiquetas en el head del HTML para optimizar el SEO.

🌟🌟 Funcionalidad esperada:

Interactividad Completa:

- La página debe permitir al usuario ver productos, añadirlos al carrito, editar el carrito, y simular la compra.

Formulario de Contacto:

- Implementa un formulario funcional que envíe datos a través de Formspree.

Diseño Responsivo:

- Asegúrate de que el diseño sea adaptable a diferentes tamaños de pantalla.

Persistencia del Carrito:

El carrito debe mantenerse activo incluso si el usuario cierra o actualiza la página, usando `localStorage` o `sessionStorage`.